Batch: B2	Roll No.: 16010124107
Experiment No. 05	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

TITLE: Implementation of Basic Process management algorithms - Preemptive (SRTF)

AIM: To implement basic Process management algorithms (SRTF)

Expected Outcome of Experiment:
CO 2. To understand the concept of process, thread and resource management.

Books/ Journals/ Websites referred:

1. **Silberschatz A., Galvin P., Gagne G. "Operating Systems Principles", Willey Eight edition.**
2. **Achyut S. Godbole , Atul Kahate "Operating Systems" McGraw Hill Third**
3. **Edition.**
4. **William Stallings, "Operating System Internal & Design Principles", Pearson.**
5. **Andrew S. Tanenbaum, "Modern Operating System", Prentice Hall.**

Pre Lab/ Prior Concepts:

Most systems handle numerous processes with short CPU bursts interspersed with I/O requests and a few processes with long CPU bursts. To ensure good time-sharing performance, a running process may be preempted to allow another to run. The ready list, or run queue, maintains all processes ready to run and not blocked by I/O or other system requests. Entries in this list point to the process control block, which stores all process information and state.

When an I/O request completes, the process moves from the waiting state to the ready state and is placed on the run queue. The process scheduler, a key component of the operating system, decides whether the current process should continue running or if another should take over. This decision is triggered by four events:

1. The current process issues an I/O request or system request, moving it from running to waiting.
2. The current process terminates.
3. A timer interrupt indicates the process has run for its allotted time, moving it from running to ready.

4. An I/O operation completes, moving the process from waiting to ready, potentially preempting the current process.

The scheduling algorithm, or policy, determines the sequence and duration of process execution, a complex task given the limited information about ready processes.

Description of the application to be implemented:

Shortest Remaining Time First Algorithm :

Shortest Remaining Time First (SRTF) is a preemptive CPU scheduling algorithm and a variant of Shortest Job First (SJF). In SRTF, the process with the smallest remaining burst time is selected for execution at every moment. If a new process arrives with a shorter burst time than the remaining time of the current process, the CPU is preempted. This approach minimizes average waiting time but may cause starvation for longer processes.

```
#include <iostream>

#include <vector>

using namespace std;

struct Process {

    int pid, at, bt, rt, ct, wt, tat;

};

int main() {

    int n;

    cout << "Enter number of processes: ";
```

```
    cin >> n;



    vector<Process> p(n);

    for(int i = 0; i < n; i++) {

        p[i].pid = i + 1;

        cout << "Enter Arrival Time for P" << i+1 << ": ";

        cin >> p[i].at;

        cout << "Enter Burst Time for P" << i+1 << ": ";

        cin >> p[i].bt;

        p[i].rt = p[i].bt;

    }

    int completed = 0, currentTime = 0;

    float total_wt = 0, total_tat = 0;



    while(completed < n) {

        int idx = -1;
```

```
        int minRT = 1e9;



        for(int i = 0; i < n; i++) {

            if(p[i].at <= currentTime && p[i].rt > 0 && p[i].rt < minRT)
{

                minRT = p[i].rt;

                idx = i;

            }

        }

        if(idx != -1) {

            p[idx].rt--;

            currentTime++;

            if(p[idx].rt == 0) {

                p[idx].ct = currentTime;

                p[idx].tat = p[idx].ct - p[idx].at;

                p[idx].wt = p[idx].tat - p[idx].bt;
```

```
                total_wt += p[idx].wt;

                total_tat += p[idx].tat;

                completed++;

            }

        } else {

            currentTime++;

        }

    }

    cout << "\nP\tAT\tBT\tCT\tWT\tTAT\n";

    for(int i = 0; i < n; i++) {

        cout << "P" << p[i].pid << "\t" << p[i].at << "\t" << p[i].bt << "\t"

             << p[i].ct << "\t" << p[i].wt << "\t" << p[i].tat << endl;

    }

    cout << "\nAverage Waiting Time: " << total_wt/n;

    cout << "\nAverage Turnaround Time: " << total_tat/n;
```

```
    return 0;


}
```

Output:

```
Enter number of processes: 5
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 5
Enter Arrival Time for P2: 0
Enter Burst Time for P2:
6
Enter Arrival Time for P3: 2
Enter Burst Time for P3: 4
Enter Arrival Time for P4: 3
Enter Burst Time for P4: 8
Enter Arrival Time for P5: 5
Enter Burst Time for P5: 10

P       AT      BT      CT      WT      TAT
P1      0       5       5       0       5
P2      0       6       15      9       15
P3      2       4       9       3       7
P4      3       8       23      12      20
P5      5       10      33      18      28

Average Waiting Time: 8.4
Average Turnaround Time: 15
```

Conclusion:

In conclusion, the Shortest Remaining Time First (SRTF) algorithm efficiently minimizes average waiting and turnaround times by always selecting the process with the least remaining burst time. Its preemptive nature allows quick handling of short

processes, but it can lead to starvation for longer processes and requires continuous monitoring of arriving processes.

Post Lab Descriptive Questions

1. Consider three processes, all arriving at time zero, with total execution time of 10, 20 and 30 units, respectively. Each process spends the first 20% of execution time doing I/O, the next 70% of time doing computation, and the last 10% of time doing I/O again. The operating system uses a shortest remaining compute time first scheduling algorithm and schedules a new process either when the running process gets blocked on I/O or when the running process finishes its compute burst. Assume that all I/O operations can be overlapped as much as possible. For what percentage of time does the CPU remain idle?

Time Interval	CPU Process	P1 I/O	P2 I/O	P3 I/O
0 – 2	Idle	I/O 2	I/O 4	I/O 6
2 – 9	P1	–	I/O 2–4	I/O 6
9 – 23	P2	–	–	Ready
23 – 44	P3	–	–	–
44 – 47	Idle (waiting for last I/O completion?)	–	–	I/O 3

- **CPU idle initially:** 0–2 because all processes are doing first I/O.
- **After t = 2**, P1 is ready → CPU executes P1.
- **SRTF scheduling:** Always pick the process with the shortest remaining compute **time** among ready processes.
- **I/O can overlap**, so the CPU is rarely idle after initial wait.
- The final I/O of P3 (t = 44 – 47) does not block CPU, but if all CPU bursts are done, it doesn't matter; CPU could be idle if nothing is left to execute.

2. Why does SRTF give better response time than SJF? Explain with reasoning.
3. The following processes are scheduled using the SRTF process scheduling policy . Determine the average waiting time and turn around time.

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9

P4	3	5

CPU idle percentage:

$$\text{Idle }\% = \frac{2}{47} \times 100 = 4.26\%$$

Date: 01/04/2026 **Signature of faculty in-charge**